프로그램 업그레이드

게임프로그래밍

2026/11/13

2022664017 **박성진**

목차

01

원본 리소스 소개

02

원본 코드의 오류

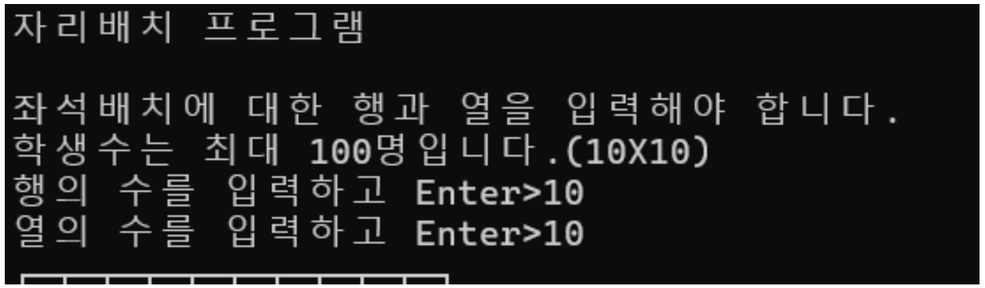
03

개선 내용 소개
오류 수정&업그레이드

04

마무리

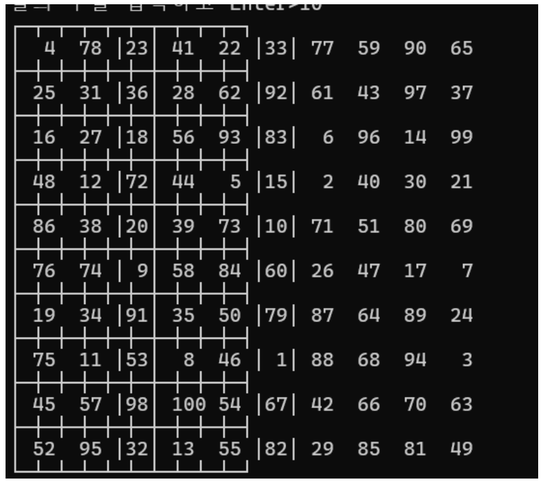
1. 원본 프로그램 소개



자리 배치 프로그램

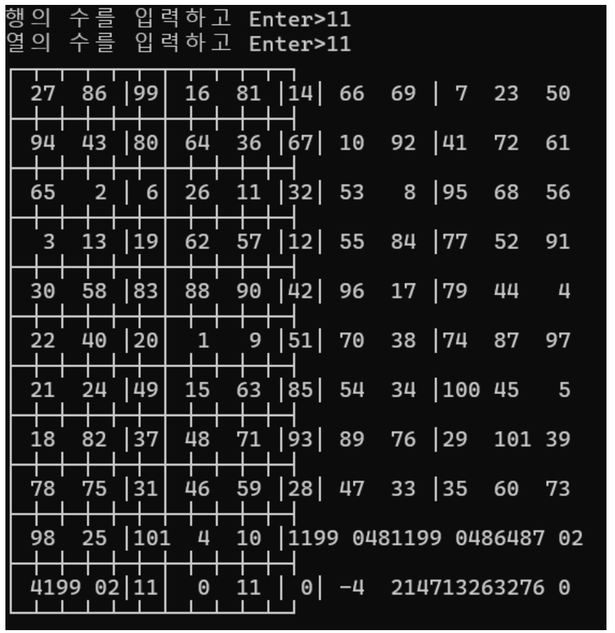
좌석 배치를 할 수 있는 프로그램.
입력에는 행, 열에 10까지의 숫자 중 하나를
적어서 출력에는 행x열 만큼의 수를 가진
학생들의 자리를 만들어 숫자를 랜덤으로 만들어
무작위로 배치해주는 프로그램.

2. 원본 코드의 오류



첨부한 이미지와 같이, 행과 열은 최대 10개까지 입력할 수 있으며, 자리를 나타내는 모서리와 변을 표현한 특수문자의 길이가 숫자의 간격과 안 맞는 정렬 오류 존재.

또한 10을 초과하는 숫자를 넣는 경우 오른쪽의 이미지와 같이 출력 값이 깨짐.



개선 내용 소개 – 목록 소개

[오류 수정]

1. 입력값 유효성 문제시 오류 수정
2. 숫자와 특수문자 간격 조정

[기능 추가]

3. 재추첨 기능
4. 추가 메뉴 확장
5. 재실행
6. 자리 고정
7. 번호 제외
8. 자리 맞바꾸기
9. 결과를 텍스트로 저장
10. 색상 강조
11. 시작 화면 추가
12. 도움말 추가
13. 소리 추가
14. 셔플 애니메이션
15. 버튼 커서형 선택
16. 가운데 정렬
17. 종료/재실행 확인
18. 번호로 자리 조회
19. 발표자 뽑기
20. 추첨 횟수 표시
21. 콘솔 창 제목 설정
22. 되돌리기
23. 직전 배치 비교
24. 랜덤 조 편성
25. 한 칸 이동
26. 이름&성별 직접 지정
27. 랜덤 시드
28. 자동 재추첨 반복
29. 추첨 이력 보기
30. 분단 구분선

3. 개선 내용 소개(오류 수정)

```c
int main(void)
{
    int row, column;

    printf("자리배치 프로그램\n\n");
    printf("좌석배치에 대한 행과 열을 입력해야 합니다.\n");
    printf("학생수는 최대 100명입니다.(10X10)\n");

    do {   //행의 입력값이 1~10을 초과한 경우에 대한 코드 추가.
        printf("행의 수를 입력하고 Enter>");
        scanf("%d", &row);

        if (row < 1 || row > 10)
            printf("오류! 행은 1~10 사이로 입력하세요.\n");

    } while (row < 1 || row > 10);

    do {   //열의 입력값이 1~10을 초과한 경우에 대한 코드 추가.
        printf("열의 수를 입력하고 Enter>");
        scanf("%d", &column);

        if (column < 1 || column > 10)
            printf("오류! 열은 1~10 사이로 입력하세요.\n");

    } while (column < 1 || column > 10);

    seat_arrangement(row, column);

    printf("\n\n\n");
    return 0;
}
```

```
자리배치 프로그램

좌석배치에 대한 행과 열을 입력해야 합니다.
학생수는 최대 100명입니다.(10X10)
행의 수를 입력하고 Enter>11
오류! 행은 1~10 사이로 입력하세요.
행의 수를 입력하고 Enter>
```

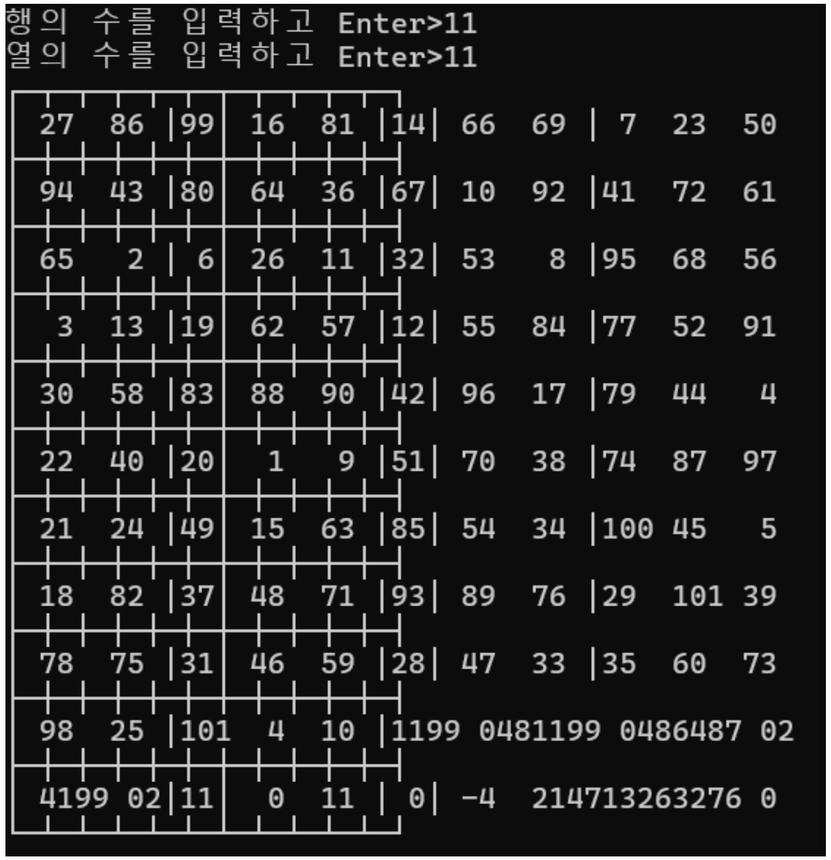
01 입력값 유효성 문제 오류 수정

오류: 기존 코드는 행과 열을 10 이상 입력/숫자가 아닌 문자를 입력시 출력값이 깨짐.

원인: 현재 main 코드에 입력 값을 검사하는 코드가 따로 없음.

개선 방향: do/while/if문을 활용해 행과 열이 1~10을 초과하는 경우에 오류 메시지를 띄우도록 추가해 해결.

3. 개선 내용 소개(오류 수정)



02 숫자와 특수문자 간격 조정

오류: 숫자가 출력되는 칸과, 모서리를 표현하는 특수문자가 출력되는 값이 간격이 맞지 않음.

원인: 숫자를 표현하는 gotoxy는 4칸 간격으로 숫자를 출력하지만, 모서리 특수문자는 printf(“%c%c” … 로 2바이트를 차지하게 출력하는데, 문제는 여기서 사용한 특수문자인 ㅡ, ┌, ├ 등은 Window 콘솔에서는 이 특수문자가 총 2칸밖에 차지하지 않게 됨.
단 이것은 오류라기 보다는 콘솔마다의 환경이 달라서 생기는 현상에 가깝다.

3. 개선 내용 소개(오류 수정)

```c
void draw_check02(int c, int r)
{
    int i, j;

    // 위쪽 테두리
    for(j = 0; j < c; j++)
        printf("+----");
    printf("+\n");

    for(i = 0; i < r; i++)
    {
        // 좌석 내부
        for(j = 0; j < c; j++)
            printf("|    ");
        printf("|\n");

        // 좌석 아래쪽 테두리
        for(j = 0; j < c; j++)
            printf("+----");
        printf("+\n");
    }
}
```

02 숫자와 특수문자 간격 조정

개선 방향: 모서리를 표현하는 방법을 특수문자 보다는 어느 환경에서든 거의 영향을 받지 않는 아스키 코드를 활용.

모서리를 출력하는 함수인 draw_check 에서 테두리 특수문자를 아스키를 지원하는 +와 -, |로 바꿔서 출력 자리가 깔끔하게 보이도록 수정.

```
+----+----+----+----+----+----+----+----+----+----+
|  4 | 78 | 23 | 41 | 22 | 33 | 77 | 59 | 90 | 65 |
+----+----+----+----+----+----+----+----+----+----+
| 25 | 31 | 36 | 28 | 62 | 92 | 61 | 43 | 97 | 37 |
+----+----+----+----+----+----+----+----+----+----+
| 16 | 27 | 18 | 56 | 93 | 83 |  6 | 96 | 14 | 99 |
+----+----+----+----+----+----+----+----+----+----+
| 48 | 12 | 72 | 44 |  5 | 15 |  2 | 40 | 30 | 21 |
+----+----+----+----+----+----+----+----+----+----+
| 86 | 38 | 20 | 39 | 73 | 10 | 71 | 51 | 80 | 69 |
+----+----+----+----+----+----+----+----+----+----+
| 76 | 74 |  9 | 58 | 84 | 60 | 26 | 47 | 17 |  7 |
+----+----+----+----+----+----+----+----+----+----+
| 19 | 34 | 91 | 35 | 50 | 79 | 87 | 64 | 89 | 24 |
+----+----+----+----+----+----+----+----+----+----+
| 75 | 11 | 53 |  8 | 46 |  1 | 88 | 68 | 94 |  3 |
+----+----+----+----+----+----+----+----+----+----+
| 45 | 57 | 98 | 100| 54 | 67 | 42 | 66 | 70 | 63 |
+----+----+----+----+----+----+----+----+----+----+
| 52 | 95 | 32 | 13 | 55 | 82 | 29 | 85 | 81 | 49 |
+----+----+----+----+----+----+----+----+----+----+
```

3. 개선 내용 소개(기능 추가)

```
do
{

    shuffle_number(sit_number, total);

    gotoxy(1, 7);
    draw_check02(column, row);

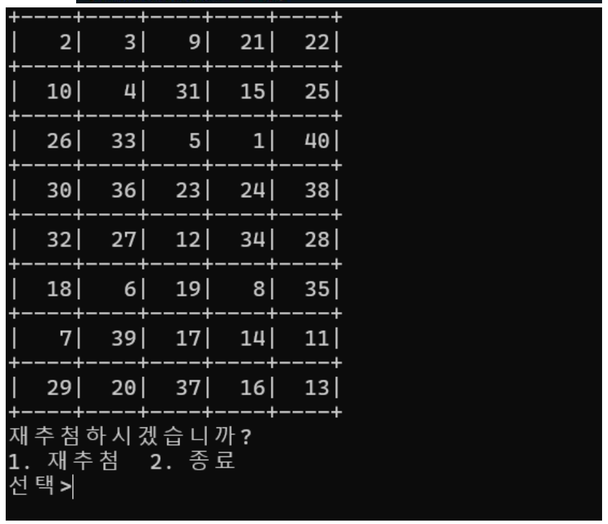
    count = 0;

    for(i = 1; i <= row; i++)
    {
        for(j = 1; j <= column; j++)
        {
            gotoxy(j * 5 - 1, 6 + i * 2);
            printf("%2d", sit_number[count]);
            count++;
        }
    }

    printf("\n\n재추첨하시겠습니까?\n");
    printf("1. 재추첨  2. 종료\n");
    printf("선택>");

    scanf("%d", &choice);

} while(choice == 1);
```



03 재추첨 기능

설명: 자리 배치 완료 후 다시 추첨 가능

구현 방식: do/while 조건문을 활용하고, 자리 배치 완료 후
“재추첨하시겠습니까? 1: 재추첨 / 2: 종료”의 텍스트로 선택지를 제공하게끔 해서, 1번을 누를 시 다시 한번 추첨되게끔 구현함.

3. 개선 내용 소개(기능 추가)

```c
printf("\n\n추가 기능 선택\n");
printf("1: 재추첨  2: 재실행  3: 종료\n");
printf("선택>");

if(scanf("%d", &choice) != 1)
{
    printf("잘못된 입력입니다.\n");
    while(getchar() != '\n');
    continue;
}

if(choice == 1)
{
    continue;
}
else if(choice == 2)
{
    return 1;
}
else if(choice == 3)
{
    return 0;
}
else
{
    printf("잘못된 선택입니다. 다시 입력하세요.\n");
```

```
추가 기능 선택
1: 재추첨   2: 재실행   3: 종료
선택>
```

04 추가 메뉴 확장

설명: 재추첨 여부만 물어보던 후속 메뉴를 확장하여 향후 코드 수정에 편리하게 함.

구현 방식: 앞서 3번에서 개선한 자리 배치 후 “재추첨하시겠습니까?” 라고 묻고 2개의 선택지만 주던 걸, 범용적 질문인 “추가 기능 선택”으로 대사를 바꾼 뒤, 입력 번호마다 다른 리턴값을 반환.

3. 개선 내용 소개(기능 추가)

```
    } while(column < 1 || column > 10);

    restart = seat_arrangement(row, column);

    if(restart == 0)
        break;
}
```

```
|  60|  43|   8|  31|  18|  11|
+----+----+----+----+----+----+
추가 기능 선택
1: 재추첨   2: 재실행   3: 종료
선택>2
행의 수를 입력하고 Enter>5
열의 수를 입력하고 Enter>6
```

05 재실행

설명: 프로그램 종료의 번거로움 없이 빠르게 재실행 가능.

구현 방식: seat_arrangement()를 int 정수형으로 바꾸고, “재실행 “ 선택시 반환값을 1을 받아 재실행 여부를 결정. 0을 받을 경우 종료.

3. 개선 내용 소개(기능 추가)

```
void shuffle_number(int sit_number[], int total, int fixed_index)
{
    int i, rnd, temp;

    for(i = total - 1; i > 0; i--)
    {
        // 고정된 좌석은 건너뜀
        if(i == fixed_index)
            continue;

        // 고정된 위치가 선택되지 않도록 함
        do {
            rnd = rand() % (i + 1);
        } while(rnd == fixed_index);

        temp = sit_number[i];
        sit_number[i] = sit_number[rnd];
        sit_number[rnd] = temp;
    }
}
```

```
자리 배치 결과 (10행 5열)
고정 번호: 26번 (4행 3열)
```

```
행의 수를 입력하고 Enter>5
열의 수를 입력하고 Enter>6

고정할 번호를 입력하세요 (1~30): 23
고정할 행을 입력하세요 (1~5): 6
고정할 열을 입력하세요 (1~6): 3
```

06 자리 고정

설명: 고정할 자리를 선 입력 가능. 이 자리는 셔플에서 제외.

구현 방식: 번호를 섞는 shuffle_number에 int fixed_index(고정된 좌석의 배열)도 전달하게끔 하고,
함수 내용에서는 고정 자석을 미리 지정하고 이 좌석은 셔플에서 건너뛰도록 구현.

3. 개선 내용 소개(기능 추가)

```
// 제외할 번호 입력 (0이면 제외 안 함)
while(1)
{
    exclude_number = input_int_range("제외할 숫자를 입력하시오(0=제외 안 함)", 0, total);

    if(exclude_number != 0 && exclude_number == fixed_number)
    {
        printf("오류! 고정된 번호는 제외할 수 없습니다.\n");
        continue;
    }
    break;
}

// 제외 번호를 빈 자리(0)로 바꿈
if(exclude_number != 0)
{
    for(i = 0; i < total; i++)
    {
        if(sit_number[i] == exclude_number)
        {
            sit_number[i] = 0;
            break;
        }
    }
}
```

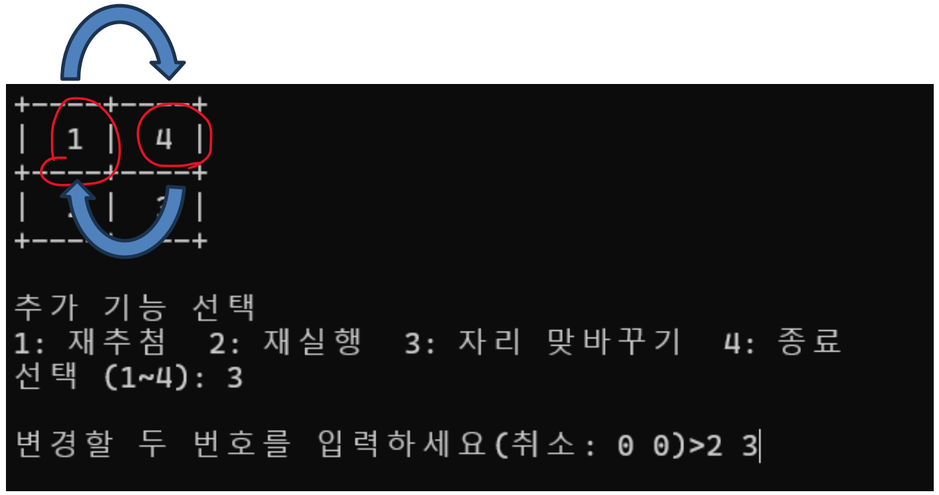
```
+----+----+----+----+----+----+
|    | 26 |  8 | 10 | 30 | 20 |
+----+----+----+----+----+----+
| 42 | 22 | 15 | 17 | 19 | 54 |
+----+----+----+----+----+----+
| 44 | 33 | 16 | 13 | 47 | 11 |
+----+----+----+----+----+----+
| 14 | 50 | 46 | 52 | 41 |  4 |
+----+----+----+----+----+----+
| 57 | 29 | 43 | 48 | 55 | 56 |
+----+----+----+----+----+----+
| 35 |  2 | 40 | 59 | 31 | 21 |
+----+----+----+----+----+----+
|  3 | 36 | 34 | 27 |  7 | 39 |
+----+----+----+----+----+----+
| 37 | 38 | 32 |  5 | 24 |  1 |
+----+----+----+----+----+----+
```

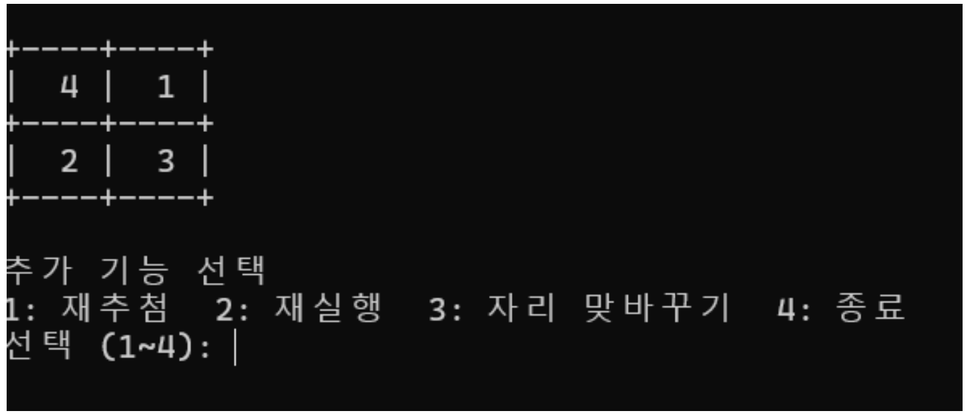
07 번호 제외

설명: 제외할 수 있는 번호 선택 가능.

구현 방식: 기존의 자리를 배치하는 구간에서 번호를 제외하는 경우를 exclude_number 함수로 추가하여, 제외할 번호를 추가로 입력할수 있게끔 함. 제외된 번호는 건너뛰어지며, 빈 자리가 하나 생기게 된다.

3. 개선 내용 소개(기능 추가)





08 자리 맞바꾸기

설명: 완료된 배치에서 나머지는 건드리지 않고 두 학생이 서로 자리를 맞바꿀수 있는 기능.

3. 개선 내용 소개(기능 추가)

```
while(1)
{
    if(need_shuffle)
        shuffle_number(sit_number, total, fixed_index);
    need_shuffle = 1;

    system("cls");

    printf("자리 배치 결과 (%d행 %d열)\n", row, column);
    printf("고정 번호: %d번 (%d행 %d열)\n",
           fixed_number, fixed_row, fixed_column);
```

```
    printf("\n\n\n추가 기능 선택\n");
    printf("1: 재추첨  2: 재실행  3: 자리 맞바꾸기  4: 종료\n");
    choice = input_int_range("선택", 1, 4);

    if(choice == 1)
        continue;
    else if(choice == 2)
        return 1;
    else if(choice == 3)
    {
        swap_seats(sit_number, total, fixed_number);
        need_shuffle = 0;       // 다음 출력 때는 셔플하지 않음
    }
    else
        return 0;
}
```

08 자리 맞바꾸기

구현 방식: 새로운 swap_seat 함수가 배열 안의 두 값 위치를 찾아서 교환해 배치를 바꿈.
이후 need_shuffle를 0으로 세팅하고, 자리 섞기 코드로 복귀.

자리 섞기 코드의 경우 need_shuffle=0일 경우에 이 코드를 통째로 건너뛰게 해서 다른 사람들은 현재 고정된 자리를 유지하고, 바꾼 두 사람만 바뀐 자리가 적용되게끔 함.

3. 개선 내용 소개(기능 추가)

```
+----+----+
|  1 |  2 |
+----+----+
|  5 |  4 |
+----+----+
|  6 |  3 |
+----+----+

추가 기능 선택
1: 재추첨   2: 재실행   3: 자리 맞바꾸기   4: 파일 저장   5: 종료
선택 (1~5): 4

seat_result.txt 파일에 저장했습니다.
계속하려면 아무 키나 누르십시오 . . .
```

```
자리 배치 결과 (3행 2열)
고정 번호: 1번 (1행 1열)
제외 번호: 없음

+----+----+
| 1 | 2 |
+----+----+
| 5 | 4 |
+----+----+
| 6 | 3 |
+----+----+
```

09 결과를 텍스트로 저장

설명: 배치 결과를 텍스트로 저장하여, 이후 언제든 볼 수 있게 함.

3. 개선 내용 소개(기능 추가)

```
printf("\n\n\n추가 기능 선택\n");
printf("1: 재추첨  2: 재실행  3: 자리 맞바꾸기  4: 파일 저장  5: 종료\n");
choice = input_int_range("선택", 1, 5);

if(choice == 1)
    continue;
else if(choice == 2)
    return 1;
else if(choice == 3)
{
    swap_seats(sit_number, total, fixed_number);
    need_shuffle = 0;
}
else if(choice == 4)
{
    save_result(sit_number, row, column,
        fixed_number, fixed_row, fixed_column, exclude_number);
    system("pause");        // 저장 메시지가 cls로 바로 지워지지 않게 잠깐 멈춤
need_shuffle = 0;       // 저장 후 다시 그릴 때 셔플하지 않음
```

```
void save_result(int sit_number[], int row, int column,
                 int fixed_number, int fixed_row, int fixed_column,
                 int exclude_number)
{
    FILE *fp;
    int i, j, count = 0;

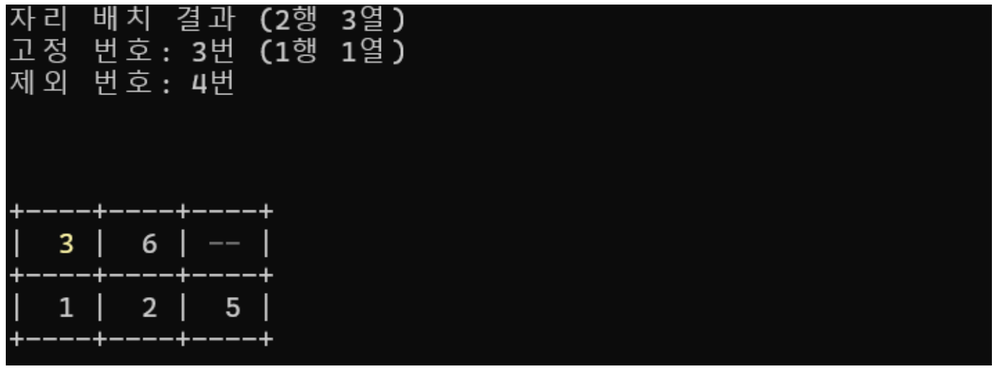
    fp = fopen("seat_result.txt", "w");
    if(fp == NULL)
    {
        printf("\n파일을 열 수 없습니다.\n");
        return;
    }

    // 머리말,,
    fprintf(fp, "자리 배치 결과 (%d행 %d열)\n", row, column);
    fprintf(fp, "고정 번호: %d번 (%d행 %d열)\n", fixed_number, fixed_row, fixed_colum
```

09 결과를 텍스트로 저장

구현 방식: 저장용인 save_result 함수를 추가해 결과를 적는 코드를 만들고,
실제 아웃풋의 추가 메뉴에서는 새로운 항목인 “파일 저장” 을 선택시 바로 이 함수가 돌아가서 새로운 txt 파일을 만들고, 그 안에 내용을 저장할수 있게끔 함.

3. 개선 내용 소개(기능 추가)



10 색상 강조

앞서 개선한 내용: 자리 고정이나 제외 등의 기능을 추가했지만, 이런 자리를 구별할 방법이 없었음.

설명: 고정 자리나 제외 자리의 색을 강조.
고정된 자리는 강조되게끔 노란색으로, 제외된 자리는 더 이상 빈 칸이 아니라 회색의 – 로 표시됨.

3. 개선 내용 소개(기능 추가)

```c
#include <stdio.h>
#include <time.h>
#include <windows.h>
#define COLOR_DEFAULT 7     // 기본 (밝은 회색)
#define COLOR_GRAY    8     // 어두운 회색
#define COLOR_YELLOW  14    // 노란색u

void set_color(int color);
```

```c
void set_color(int color)
{
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
}
```

```c
    gotoxy(1, 7);
    draw_check02(column, row);

            count = 0;

    for(i = 1; i <= row; i++)
    {
        for(j = 1; j <= column; j++)
        {
            gotoxy(j * 5 - 2, 6 + i * 2);

            if(sit_number[count] == 0)
            {
                set_color(COLOR_GRAY);          // 제외된 자리
                printf("--");
            }
            else if(sit_number[count] == fixed_number)
            {
                set_color(COLOR_YELLOW);        // 고정 번호
                printf("%2d", sit_number[count]);
            }
            else
            {
                printf("%2d", sit_number[count]);
            }

            set_color(COLOR_DEFAULT);           // 반드시 원래 색으로 복구
            count++;
        }
    }
```

10 색상 강조

구현 방식: 색을 입히기 위해 define로 색을 정의하고, set_color 함수로 색을 입힐수 있게 한다. 그리고 고정되고 제외된 좌석에는 set_color 함수를 호출하여 색을 입힌다.

3. 개선 내용 소개(기능 추가)

```c
int start_screen(void)
{
    int choice;

    while(1)
    {
        system("cls");

        printf("==============================\n");
        printf("        자리 배치 프로그램\n");
        printf("==============================\n\n");
        printf("   1: 실행\n");
        printf("   2: 도움말\n");
        printf("   3: 종료\n\n");

        choice = input_int_range("선택", 1, 3);

        if(choice == 1)
        {
            system("cls");
            return 1;          // 프로그램 시작
        }
        else if(choice == 2)
            show_help();       // 도움말을 보고 돌아오면 시작 화면이 다시 뜸
        else
            return 0;          // 종료
    }
}
```

```
==============================
        자리 배치 프로그램
==============================

   1: 실행
   2: 도움말
   3: 종료

선택 (1~3):
```

11 시작 화면 추가

설명: 메인 프로그램 실행 이전 시작 화면을 추가.

구현 방식: start_screen 함수를 만들었고 main 코드 기본 실행 메뉴 앞에서 우선적으로 실행되게 하였음.

이후 업그레이드 예정인 도움말, 나가기 기능 역시 이 화면에서 구현할 예정이라, 우선적으로 버튼만 추가.

3. 개선 내용 소개(기능 추가)

```c
void show_help(void)
{
    FILE *fp;
    char line[200];

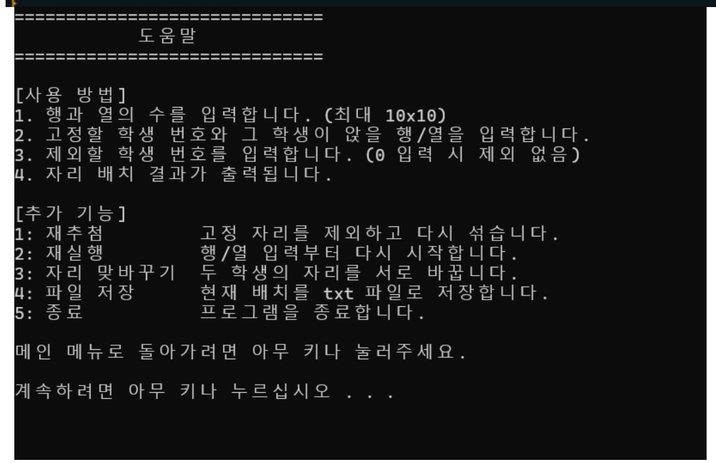
    system("cls");

    fp = fopen("help.txt", "r");
    if(fp == NULL)
    {
        printf("도움말 파일(help.txt)을 찾을 수 없습니다.\n");
        printf("실행 파일과 같은 폴더에 help.txt가 있는지 확인하세요.\n\n");
        system("pause");
        return;
    }

    while(fgets(line, sizeof(line), fp) != NULL)   // 한 줄씩 읽어서 그대로 출력
        printf("%s", line);

    fclose(fp);

    printf("\n\n");
    system("pause");
}
```



12 도움말 추가

설명: 앞서 11번에서 추가한 메뉴에서 2번 선택지인 도움말 선택시 도움말이 출력되도록 추가하여, 이해를 도움.

구현 방식: 도움말이 적힌 help.txt 파일을 코드와 같은 위치에 넣고, 코드에서는 help_text 함수를 만들어 그 안에
FILE *fp; / p = fopen(" help.txt " , " r "); 를 통해 호출하게 함.

*참고: txt 파일은 ANSI로 인코딩해야 글자 출력이 정상적으로 됨.

3. 개선 내용 소개(기능 추가)

```
void play_select_sound(void)
{
    Beep(800, 50);          // 짧은 "띡" 소리
}

void play_done_sound(void)
{
    Beep(523, 80);          // 도
    Beep(659, 80);          // 미
    Beep(784, 150);         // 솔  (상승음으로 완료 느낌)
}
```

13 소리 추가

설명: 시작 화면에서 실행 버튼 누를시, 배치 완료시,추가 버튼 누를시 세 경우에서 소리가 들리도록 함.

구현 방식: play_select_sound, play_done_sound 두 함수를 추가하고, Beep로 소리를 들리게 한 다음, 위의 경우에서 이 함수를 호출하게 함.

3. 개선 내용 소개(기능 추가)

14 셔플 애니메이션

```
void shuffle_animation(int sit_number[], int total, int fixed_index,
                       int row, int column, int fixed_number)
{
    int frame;

    system("cls");
    printf("추첨 중입니다...\n");

    gotoxy(1, 7);
    draw_check02(column, row);          // 격자는 한 번만 그림

    for(frame = 0; frame < ANIM_FRAMES; frame++)
    {
        shuffle_number(sit_number, total, fixed_index);
        print_numbers(sit_number, row, column, fixed_number);
        Sleep(40 + frame * 12);         // 갈수록 느려지는 룰렛
    }

    Sleep(300);                         // 결과 확정 전 잠깐 멈춤
}
```

설명: 자리가 배치될때, 애니매이션이 재생되어 좀더 생생하게 출력되게 함.

구현 방식: shuffle_animation 함수를 추가하고, 숫자가 확정되기 전까지 기존의 섞기 함수인 shuffle_number을 연속 15번 호출해 룰렛을 돌리는 것처럼 표현.
Sleep(40 + frame * 12)로 점점 느려지는 느낌을 연출.

3. 개선 내용 소개(기능 추가)

```c
int select_menu(const char *items[], int count, int x, int y)
{
    int cursor = 0;
    int i, key;

    set_cursor_visible(0);

    // 항목 글자는 한 번만 출력 (▷가 들어갈 자리를 비우려고 x+3에서 시작)
    for(i = 0; i < count; i++)
    {
        gotoxy(x + 3, y + i);
        printf("%s", items[i]);
    }

    while(1)
    {
        // ▷ 기호 그리기: 선택된 줄만 ▷, 나머지는 공백으로 지움
        for(i = 0; i < count; i++)
        {
            gotoxy(x, y + i);
            if(i == cursor)
                printf("▷ ");
            else
                printf("  ");
        }

        key = _getch();

        if(key == 224 || key == 0)        // 방향키는 2바이트라 한 번 더 읽음
        {
            key = _getch();

            if(key == KEY_UP)
                cursor = (cursor - 1 + count) % count;   // 맨 위에서 위로 가면 맨 아래로
            else if(key == KEY_DOWN)
                cursor = (cursor + 1) % count;
        }
        else if(key == KEY_ENTER)
        {
            play_select_sound();          // 앞에서 만든 선택 소리 (없으면 이 줄 삭제)
            gotoxy(1, y + count + 1);     // 이후 출력이 메뉴를 덮지 않게 커서를 아래로
            set_cursor_visible(1);
            return cursor;
        }
    }
}
```

```
==============================
        자리 배치 프로그램
==============================

  ▷ 실행
    도움말
    종료
```

15 버튼 커서형 선택

설명: 제공되는 버튼들을 이제 1,2,3 등의 숫자 누르기 방식이 아닌, ▷ 모양의 커서를 움직이게 함.

구현 방식: getch()로 방향키를 즉석에서 읽게 하고, ▷ 커서를 읽은 줄에서 출력되고 나머지는 공백으로 출력되게 한다.

3. 개선 내용 소개(기능 추가)

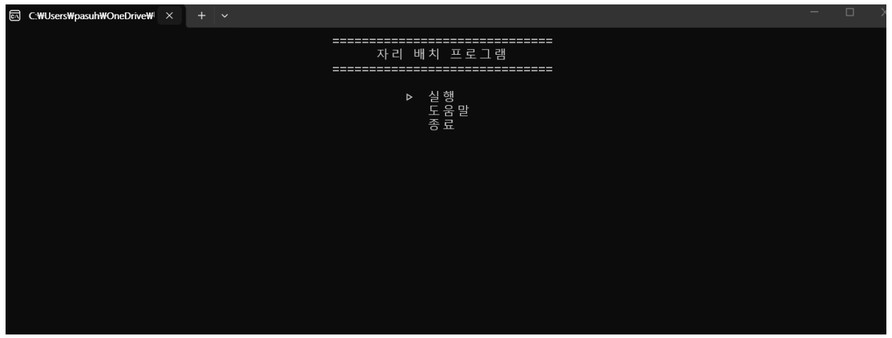
```c
// 현재 보이는 콘솔 창의 가로 칸 수
int get_console_width(void)
{
    CONSOLE_SCREEN_BUFFER_INFO csbi;

    GetConsoleScreenBufferInfo(GetStdHandle(STD_OUTPUT_HANDLE), &csbi);
    return csbi.srWindow.Right - csbi.srWindow.Left + 1;
}

// 가로 width칸짜리 내용을 가운데에 놓으려면 시작해야 할 x좌표 (1부터)
int center_x(int width)
{
    int x = (get_console_width() - width) / 2 + 1;

    if(x < 1)
        x = 1;          // 창이 내용보다 좁으면 왼쪽 끝에서 시작
    return x;
}

// y번째 줄에 글자를 가운데 정렬로 출력
void print_center(int y, const char *text)
{
    gotoxy(center_x((int)strlen(text)), y);
    printf("%s", text);
}
```



16 가운데 정렬

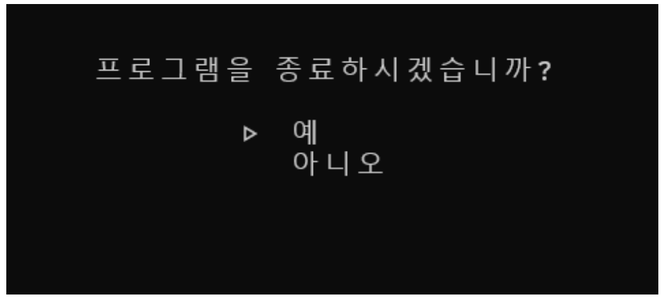
설명: 전체 화면에서 화면이 가운데 정렬되어 깔끔하게 보인다.

구현 방식: getConsoleScreenBufferInfo()로 현재 글자 출력하는 기본 콘솔 창을 호출해 메모리 주소에 저장해 균형을 맞춘다.

3. 개선 내용 소개(기능 추가)

```
else if(choice == 2)
    if(confirm("처음부터 다시 시작하시겠습니까?"))
    return 1;
    need_shuffle = 0;          // "아니오"면 섞지 않고 화면 복귀
```

```
else
    if(confirm("프로그램을 종료하시겠습니까?"))
        return 0;
    need_shuffle = 0;
    }
}
```



17 종료/재실행 확인

설명: 종료나 재실행시 바로 실행하지 않고 한번더 물어보게 해서 실수를 막음.

구현 방식: 종료나 재실행 전 텍스트 추가, 선택지를 띄우는 함수 추가.

3. 개선 내용 소개(기능 추가)

```c
void find_seat(int sit_number[], int total, int column)
{
    int n, i, found = 0;

    n = input_int_range("조회할 번호를 입력하세요", 1, total);

    for(i = 0; i < total; i++)
    {
        if(sit_number[i] == n)
        {
            printf("\n%d번 학생의 자리: %d행 %d열\n",
                   n, i / column + 1, i % column + 1);
            found = 1;
            break;
        }
    }

    if(!found)
        printf("\n%d번은 제외된 번호라 자리가 없습니다.\n", n);

    system("pause");
```



18 번호로 자리 조회

설명: 번호 입력시 그 번호가 몇 행 몇 열에 있는지 알려줌.

구현 방식: 추가 메뉴에서 자리 조회 선택지 추가, 새로운 find_seat 함수에서 i / column + 1, i % column + 1 로 각각 행과 열을 찾는다.

3. 개선 내용 소개(기능 추가)

```c
void pick_presenter(int sit_number[], int total, int column)
{
    int i, count = 0, n = 0, target;

    for(i = 0; i < total; i++)          // ① 배치된 학생 수 세기 (빈 자리 0 제외)
        if(sit_number[i] != 0)
            count++;

    target = rand() % count;            // ② 몇 번째 학생을 뽑을지 결정

    for(i = 0; i < total; i++)          // ③ 그 학생의 위치 찾기
    {
        if(sit_number[i] != 0)
        {
            if(n == target)
                break;
            n++;
        }
    }

    printf("\n발표자: %d번 학생 (%d행 %d열)\n",
           sit_number[i], i / column + 1, i % column + 1);
    play_done_sound();
    system("pause");
}
```



19 발표자 지정

설명: 배치된 학생 중 1명을 발표자로 뽑아, 번호와 자리를 보여줌.

구현 방식: 새로운 pick_presenter 함수에서 학생 수를 센 후
target = rand() % count; 로 몇 번째 학생을 뽑을지 지정한다.

3. 개선 내용 소개(기능 추가)

```
if(need_shuffle)
{
    shuffle_animation(sit_number, total, fixed_index,
                      row, column, fixed_number);
    shuffled = 1;

    draw_count++;                                    // 횟수 +1
    now = time(NULL);                                // 현재 시각
    strftime(time_str, sizeof(time_str),
     "%Y-%m-%d %H:%M:%S", localtime(&now)); // 문자열로 변환
}
```

```
자리 배치 결과 (2행 2열)
고정 번호: 1번 (1행 1열)
제외 번호: 없음

1회차 추첨  (2026-11-12 10:53:12)

+----+----+
|  1 |  2 |
+----+----+
|  4 |  3 |
+----+----+
```

20 추첨 횟수 및 시각 표시

설명: 추첨 횟수와 추첨한 현재 시각을 보여줌.

구현 방식:
strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", localtime(&now));
를 통해 현재 날짜를 받고, draw_count++를 통해 현재 추첨 횟수를 받아 표시.

3. 개선 내용 소개(기능 추가)

```c
int main(void)
{
    int row, column;
    int restart = 1;

    srand((unsigned int)time(NULL));
    system("title 자리 배치 프로그램");
```

```c
sit_number[number_index] = temp;
sprintf(buf, "title 자리 배치 프로그램 - %d행 %d열", row, column);
system(buf);
```

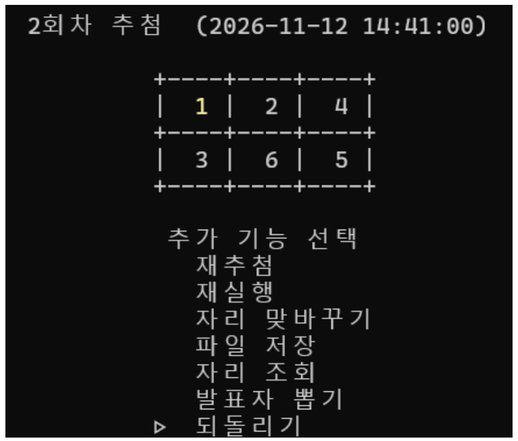


21 콘솔 창 제목 설정

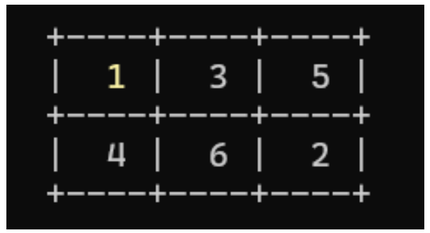
설명: 실행창 왼쪽 위에 표시되는 콘솔창의 제목에 현재 프로그램 이름과 배치시 (행, 열) 이 출력되도록 변경.

구현 방식: system(title….) 는 명령 프롬포트의 title 명령을 실행하는데, 이를 활용.

3. 개선 내용 소개(기능 추가)



되돌리기 전



되돌린 후

22 되돌리기

설명: 추첨 완료 후 추가 기능 사용 후, 최초 추첨 완료 상태로 복구.

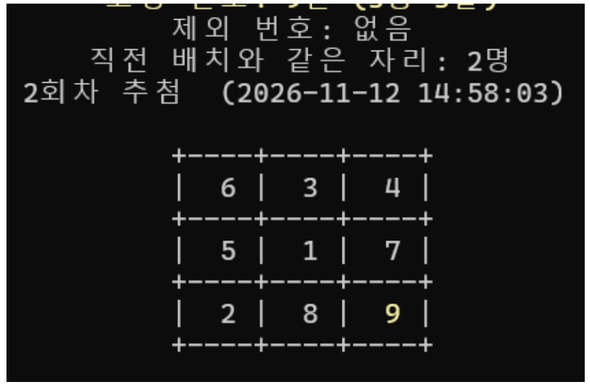
구현 방식: 추가 기능 실행 직전, 현재 배열 상태를 학생 수를 저장하는 문자열과 동일한 크기의 backup[100] 변수에 저장.

```
if(draw_count > 0)                                    // 첫 추첨 전 상태는 백업 안 함
{
    memcpy(backup, sit_number, sizeof(sit_number));
    has_backup = 1;
}

shuffle_animation(sit_number, total, fixed_index,
                  row, column, fixed_number);
shuffled = 1;
```

3. 개선 내용 소개(기능 추가)

```
if(has_backup)
{
    for(i = 0; i < total; i++)
        if(i != fixed_index && sit_number[i] != 0 && sit_number[i] == backup[i])
            same_count++;
}
```



23 직전 배치와의 비교

설명: 여러 번 재추첨시 직전 결과와 같은 자리에 앉은 학생이 몇 명인지 출력.

구현 방식: 셔플 뒤에 새로운 자리배치 결과인 sit_number[100]과 직전 자리배치 결과인 backup[100]을 서로 비교하는 계산 추가.

3. 개선 내용 소개(기능 추가)

```c
void make_teams(int sit_number[], int total)
{
    int list[100], count = 0;
    int i, j, t, k, tmp, teams;

    for(i = 0; i < total; i++)              // ① 배치된 학생만 모으기 (빈 자리 제외)
        if(sit_number[i] != 0)
            list[count++] = sit_number[i];

    teams = input_int_range("조의 수를 입력하세요", 1, count);

    for(i = count - 1; i > 0; i--)          // ② 학생 목록을 무작위로 섞기
    {
        j = rand() % (i + 1);
        tmp = list[i];
        list[i] = list[j];
        list[j] = tmp;
    }

    for(t = 0; t < teams; t++)              // ③ 번갈아 나눠주며 조별 출력
    {
        printf("\n%d조:", t + 1);
        for(k = t; k < count; k += teams)
            printf(" %d", list[k]);
    }
    printf("\n\n");

    play_done_sound();
    system("pause");
}
```

```
▷  조 편성
조의 수를 입력하세요 (1~36): 6
```

```
1조: 16 21 36 5 6 11
2조: 23 17 25 29 2 7
3조: 34 9 8 3 33 12
4조: 20 1 19 15 28 27
5조: 4 13 22 31 32 30
6조: 10 14 24 26 35 18
```

24 랜덤 조 편성

설명: 추첨 후, 조의 수를 입력하면 배치된 학생들을 조별로 다시 나눠주는 기능.

구현 방식: 빈 자리 제외 학생 번호를 모은 목록을 만든 후, 카드를 한 장씩 나눠주는 것과 같이 조의 수만큼 건너뛰며 학생 숫자를 조에 나누어줌.

3. 개선 내용 소개(기능 추가)

```c
void rotate_seats(int sit_number[], int total, int fixed_index)
{
    int list[100], count = 0;
    int i, last;

    for(i = 0; i < total; i++)              // ① 고정 자리를 뺀 값들을 순서대로 모음
        if(i != fixed_index)
            list[count++] = sit_number[i];

    if(count < 2)                           // 움직일 학생이 없으면 종료
        return;

    last = list[count - 1];                 // ② 맨 마지막 값을 기억해 두고,
    for(i = count - 1; i > 0; i--)          //    나머지를 한 칸씩 뒤로 밀고
        list[i] = list[i - 1];
    list[0] = last;                         //    마지막 값을 맨 앞에 넣음

    count = 0;
    for(i = 0; i < total; i++)              // ③ 다시 배열에 채워 넣기
        if(i != fixed_index)
            sit_number[i] = list[count++];
}
```

```
1회차 추첨  (2026-11-12 15:13:18)

+----+----+----+
|  1 |  3 |  4 |
+----+----+----+
|  6 |  7 |  5 |
+----+----+----+
|  2 |  8 |  9 |
+----+----+----+

 추가 기능 선택
   재추첨
   재실행
   자리 맞바꾸기
   파일 저장
   자리 조회
   발표자 뽑기
   되돌리기
   조 편성
 ▷ 한 칸 이동
```

```
+----+----+----+
|  8 |  1 |  3 |
+----+----+----+
|  4 |  6 |  7 |
+----+----+----+
|  5 |  2 |  9 |
+----+----+----+
```

25 한 칸 이동

설명: 배치된 자리를 오른쪽으로 한 칸씩 이동(고정 자리 제외)

구현 방식: 고정 자리를 건너뛴 채 나머지 값을 한 줄로 만든 뒤, 전체를 한 칸씩 뒤로 밀어내서(마지막 자리의 경우 맨 앞으로) 출력.

3. 개선 내용 소개(기능 추가)

```
    표시 방식을 선택하세요

  ▷ 번호로만 표시 (기존 방식)
    이름과 성별 입력
```

```
학생 4명의 이름과 성별을 입력하세요.
(이름은 한글 4자 / 영문 8자 이내, 공백 없이)

1번 이름: 박성진
성별 1:남 2:여 (1~2): 1
2번 이름: 성진박
성별 1:남 2:여 (1~2): 1
3번 이름: 박진성
성별 1:남 2:여 (1~2): 2
4번 이름: 성박진
성별 1:남 2:여 (1~2): 2
```

```
    1회차 추첨  (2026-11-12 23:03:46)
남: 하늘색  /  여: 분홍색  /  고정: 노란색
          +--------+--------+
          | 박성진 | 박진성 |
          +--------+--------+
          | 성박진 | 성진박 |
          +--------+--------+
```

26 이름&성별 직접 지정

설명: 지금까지 번호로만 보이던 학생 목록을 이제 이름과 성별을 직접 지정하고 출력 가능.

남자는 하늘색, 여자는 분홍색.

3. 개선 내용 소개(기능 추가)

```c
int cell_w(void)
{
    return g_use_names ? 9 : 5;
}

// 표시 방식 선택: 0 = 번호만, 1 = 이름 · 성별
int choose_display_mode(void)
{
    const char *items[2] = {"번호로만 표시 (기존 방식)", "이름과 성별 입력"};
    int result;

    do {
        system("cls");
        print_center(5, "표시 방식을 선택하세요");
        result = select_menu(items, 2, -1, 7);
    } while(result == -1);              // 창 크기가 바뀌면 다시 그림

    return result;
}

// 1번부터 total번까지 이름과 성별을 한 명씩 입력
void input_students(int total)
{
    int i;
    char buf[100];

    system("cls");
    printf("학생 %d명의 이름과 성별을 입력하세요.\n", total);
    printf("(이름은 한글 4자 / 영문 8자 이내, 공백 없이)\n\n");

    for(i = 1; i <= total; i++)
    {
        while(1)
        {
            printf("%d번 이름: ", i);
            scanf("%99s", buf);
            while(getchar() != '\n');   // 같은 줄에 남은 입력 버리기

            if(strlen(buf) <= 8)
                break;
            printf("오류! 이름은 한글 4자 / 영문 8자 이내로 입력하세요.\n");
        }

        strcpy(g_name[i], buf);
        g_gender[i] = input_int_range("성별 1:남 2:여", 1, 2);
    }
}
```

26 이름&성별 직접 지정

구현 방식: 학생 번호를 인덱스로 쓰는 전역 배열(g_name, g_gender)에 이름과 성별을 저장 후 시작시 "번호로만 / 이름 · 성별 입력" 중 방식을 선택하도록 함.

이름 모드에서는 칸 폭을 함수(Cell_w)로 넓혀 이름을 칸 가운데에 출력하고, 성별은 색(남 하늘색, 여 분홍색)으로 구분.

번호가 곧 인덱스라서 재추첨, 맞바꾸기 등으로 자리가 바뀌어도 이름이 번호를 따라 움직이고, 번호 모드는 기존과 동일하게 동작함.

3. 개선 내용 소개(기능 추가)

```
seed = input_int_range("랜덤 시드 (0=매번 다르게)", 0, 99999);
if(seed != 0)
    srand((unsigned int)seed);
// 제외할 번호 입력 (0이면 제외 안 함)
```

```
고정할 번호를 입력하세요 (1~4): 1
고정할 행을 입력하세요 (1~2): 1
고정할 열을 입력하세요 (1~2): 1
랜덤 시드 (0=매번 다르게) (0~99999): |
```

27 랜덤 시드 입력

설명: 추첨시 무작위가 아닌 시드값을 통해 같은 설정에서 같은 추첨 결과를 재현할수 있는 기능 추가.

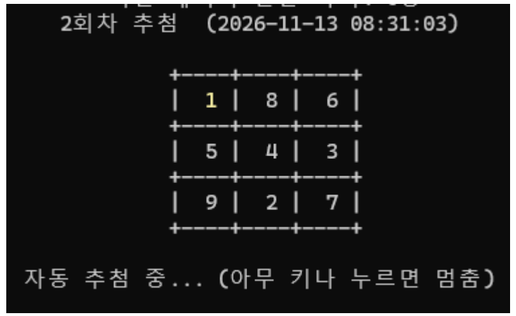
구현 방식: srand()로 시작 값에서 정해진 수열을 만듦.

3. 개선 내용 소개(기능 추가)

```
if(auto_mode)
{
    print_center(9 + row * 2, "자동 추첨 중... (아무 키나 누르면 멈춤)");

    for(t = 0; t < 30 && !_kbhit(); t++)
        Sleep(50);                  // 약 1.5초 대기, 키 입력이 오면 바로 중단

    if(_kbhit())
        {
            _getch();               // 눌린 키 버리기
            auto_mode = 0;
            need_shuffle = 0;       // 멈춘 화면 그대로 유지
        }
    continue;                       // 키가 없으면 need_shuffle=1 그대로 → 다음 추첨
}
```



28 자동 재추첨 반복

설명: 선택시 재추첨을 키를 누를 때까지 반복.

구현 방식: while(1)이 메뉴 대신 대기 후 continue하게 해서 재추첨 반복.

3. 개선 내용 소개(기능 추가)

```
void show_history(int history[][100], int hist_count, int draw_count,
                  int row, int column)
{
    int h, i, j;

    system("cls");
    printf("추첨 이력 (최근 %d회)\n", hist_count);

    for(h = 0; h < hist_count; h++)
    {
        printf("\n[%d회차]\n", draw_count - hist_count + 1 + h);

        for(i = 0; i < row; i++)
        {
            for(j = 0; j < column; j++)
            {
                if(history[h][i * column + j] == 0)
                    printf(" --");
                else
                    printf("%3d", history[h][i * column + j]);
            }
            printf("\n");
        }
    }

    printf("\n");
    system("pause");
}
```

```
추첨 이력 (최근 1회)

[1회차]
  1  2  5
  7  9  6
  8  3  4
```

29 추첨 이력 보기

설명: 최근 10회까지의 추첨 결과를 저장해 두고 한 화면에서 모아서 볼 수 있음.

구현 방식: 배치 시마다 history라는 2차원 배열에 복사본을 한 줄씩 쌓고,(10개까지. 10개 이후는 memmove로 한 칸 당김)이를 출력해준다.

3. 개선 내용 소개(기능 추가)

```c
void put_border(const char *s, int j, int c)
{
    if(g_group > 0 && j > 0 && j < c && j % g_group == 0)
        set_color(COLOR_GREEN);

    printf("%s", s);
    set_color(COLOR_DEFAULT);
}
```



30 분단 구분선

설명: 시작시 분단을 나눌 지 물어본 후, 분단 경계를 초록색으로 구분.

구현 방식: 열 경계 j가 분단 구분선인 g_group의 배수일때만 선을 초록색으로 표시.

THANK YOU

구현 갯수 30/30 감사합니다!